

Visual Exploratory Data Analysis (VisEDA): PART D

Point Cloud Exploratory Data Analysis

A Guide to 3D Point Cloud Dataset Analysis

Isaac Osei Agyemang

Version 1.0.0

2026

Preface

VisEDA was created to bring the discipline and convenience of exploratory data analysis to visual and spatial datasets. Point clouds require a specialised form of EDA because every sample is an unordered set of three-dimensional coordinates, often augmented with intensity, colour, or sensor-specific attributes. Dataset quality cannot be assessed reliably from a single 3D preview: density, scale, coordinate extent, duplication, missing values, outliers, point spacing, height structure, and geometric shape must all be examined quantitatively.

PointCloudEDA provides one unified interface for a single cloud, a list of clouds, a directory of point-cloud files, or in-memory NumPy arrays. It computes per-cloud geometry, height statistics, nearest-neighbour spacing, data-quality indicators, PCA-based shape descriptors, attribute summaries, and dataset-level distances. It also produces a comprehensive dashboard, single-cloud projections, cloud preview grids, height-distribution overlays, pairwise-distance heatmaps, and a self-contained HTML report.

Isaac Osei Agyemang

August 2026

Table of Contents

Preface	2
Table of Contents	3
Chapter 1 — Introduction	6
1.1 What is PointCloudEDA?	6
1.2 Why Point Cloud EDA Matters	6
1.3 The Point Cloud Data Model.....	6
1.4 Design Philosophy	6
1.5 Installation	7
1.6 Supported Data Formats	7
1.7 Coordinate and Attribute Conventions.....	7
Chapter 2 — Quick Start	8
2.1 Analyse a Directory	8
2.2 Analyse a Single File	8
2.3 Load NumPy Arrays Directly.....	8
2.4 Obtain the Summary	9
2.5 Generate All Visualisations.....	9
2.6 Generate an HTML Report	9
2.7 Command-Line Interface	9
2.8 Run the Complete Test Script.....	9
Chapter 3 — The PointCloudEDA Class	10
3.1 Constructor	10
3.2 load() — Files and Directories	10
3.3 load_arrays() — In-Memory Clouds	10
3.4 summary() — Nested Dataset Statistics	10
3.5 get_record() and get_cloud()	11
3.6 pairwise_cloud_distances()	11
3.7 report()	12
Chapter 4 — The PointCloudRecord Object	13
4.1 Identity and Status Fields.....	13
4.2 Coordinate and Extent Fields	13
4.3 Height and Radial Fields	13
4.4 Quality Fields	13
4.5 PCA Shape Descriptors	14
4.6 Attribute and Preview Fields	14
Chapter 5 — What Is Analysed	15
5.1 Input Validation and Finite Filtering	15
5.2 Reproducible Downsampling	15

5.3 Coordinate Statistics and Centroid	15
5.4 Bounding Box, Volume, and Density	15
5.5 Height Structure	15
5.6 Radial Spread and Outlier Fraction	15
5.7 Duplicate Fraction	16
5.8 Nearest-Neighbour Spacing	16
5.9 PCA-Based Global Shape	16
5.10 Intensity and RGB Attributes	16
5.11 Descriptor-Space Dataset Distance	16
Chapter 6 — Visualisations	17
6.1 plot_dataset() — Dataset Dashboard	17
Row 1: Dataset Overview and Label Distribution	18
Row 2: Point Count, Volume, Density, and Height Variation	18
Row 3: XYZ Spans and Quality Metrics	18
Row 4: PCA Shape Descriptors and Distance Heatmap	18
6.2 plot() — Single-Cloud Dashboard	18
Cloud Overview Card	19
3D Scatter and Orthographic Projections	19
Height Histogram and XY Density Map	19
Nearest-Neighbour Spacing	19
6.3 plot_clouds_grid() — Preview Grid	19
6.4 plot_height_distribution() — Z Distribution Overlay	19
6.5 plot_pairwise_cloud_distances() — Distance Heatmap	20
Chapter 7 — Interpreting Results and Common Findings	22
7.1 Red Flags to Review	22
7.2 Reading Shape Descriptors Together	22
7.3 Interpreting Density Carefully	22
7.4 Example Findings from the Attached Outputs	22
7.5 Thresholds Are Dataset-Dependent	23
Chapter 8 — Advanced Usage	24
8.1 Working with Large Datasets	24
8.2 Access and Filter Records	24
8.3 Export Records to pandas	24
8.4 Use PointCloudEDA Before Model Training	25
8.5 Prevent Similarity Leakage	25
8.6 Compare Two Datasets	25
8.7 Reproducibility Considerations	25
Chapter 9 — HTML Report, CLI, and Testing	26
9.1 HTML Report Structure	26

- 9.2 Command-Line Options26
- 9.3 Test Coverage26
- 9.4 Common Errors.....26
- Chapter 10 — API Reference27**
 - 10.1 Return and Failure Behaviour27
- Chapter 11 —Roadmap.....28**
 - Version 1.0.0.....28
 - Roadmap28
- Appendix A — PointCloudRecord Fields29**
- Appendix B — summary() Schema.....30**
- Appendix C — Mathematical Reference31**
- Appendix D — Glossary32**
- Appendix E — Acknowledgements33**

Chapter 1 — Introduction

1.1 What is PointCloudEDA?

PointCloudEDA is the 3D point-cloud analysis module of VisEDA. It analyses individual point clouds and complete point-cloud datasets using the same PointCloudEDA class. The first three columns of every input array are interpreted as X, Y, and Z coordinates. Additional columns are retained as attributes and may represent intensity or RGB information.

At the individual-cloud level, PointCloudEDA computes coordinate ranges, centroid, bounding-box dimensions, volume, density, height statistics, radial spread, duplicate fraction, finite fraction, outlier fraction, nearest-neighbour spacing, PCA shape descriptors, intensity statistics, and colour statistics. At the dataset level, it aggregates these records into distributions, class counts, quality summaries, geometric comparisons, and pairwise descriptor-space distances.

1.2 Why Point Cloud EDA Matters

- **Coordinate validity:** NaN and infinite XYZ rows can silently break registration, voxelisation, meshing, and neural-network pipelines.
- **Point-count variation:** Clouds with a few hundred points and clouds with millions of points require different sampling and batching strategies.
- **Scale and units:** Metres, centimetres, millimetres, and arbitrary normalised coordinates can coexist unless explicitly audited.
- **Density inconsistency:** Two clouds may describe the same object but use very different sampling densities, biasing local geometric features.
- **Duplicate points:** Repeated XYZ coordinates inflate point counts without adding information and can distort neighbour statistics.
- **Outliers:** Isolated points far from the main cloud can expand the bounding box and destabilise normalisation.
- **Shape imbalance:** A dataset may contain mostly planar, linear, or volumetric structures even when class labels appear balanced.
- **Attribute inconsistency:** Some files may contain intensity or RGB columns while others contain only XYZ.
- **Data leakage:** Near-identical clouds can appear in both training and validation sets unless descriptor similarity is checked.
- **Corrupt formats:** Malformed PLY headers, missing LAS dependencies, or inconsistent text delimiters can leave unreadable samples.

1.3 The Point Cloud Data Model

A point cloud is represented as an $N \times D$ matrix. N is the number of points and D is the number of columns. PointCloudEDA requires $D \geq 3$, with columns 0, 1, and 2 corresponding to X, Y, and Z. A one-dimensional input containing at least three values is reshaped to a single-point cloud.

$$\mathbf{P} \in \mathbb{R}^{(N \times D)}, \quad D \geq 3$$

Columns 0–2 are XYZ; later columns are optional attributes.

The implementation converts arrays to float32 and removes rows whose XYZ coordinates are not all finite. The fraction retained is recorded as `finite_fraction`. If no finite XYZ rows remain, the cloud is marked corrupt.

1.4 Design Philosophy

- **One interface for files and arrays.** `load()` handles files and directories; `load_arrays()` handles in-memory arrays.
- **Dataset-first EDA.** The implementation emphasises distributions and comparisons across clouds, not only attractive 3D previews.
- **Reproducible downsampling.** Large clouds are sampled with a fixed random seed and sorted selected indices.
- **Optional expensive analyses.** Nearest-neighbour and PCA geometry calculations can be disabled for quick runs.
- **Lazy dependencies.** `scikit-learn` and `laspy` are imported only when the corresponding features or formats are used.
- **Transparent records.** Every analysed cloud is represented by a `PointCloudRecord` containing the computed values.

- **Graceful degradation.** Missing optional dependencies produce unavailable metrics rather than preventing the remainder of the analysis.

1.5 Installation

```
# Core package
pip install viseda numpy matplotlib

# Nearest-neighbour calculations
pip install scikit-learn

# LAS / LAZ point clouds
pip install laspy
pip install "laspy[lazrs]"

# Editable Word documentation generator
pip install python-docx pillow
```

Note: LAS files can be read with laspy. LAZ decompression normally requires an additional backend such as lazrs. When `compute_neighbors=False`, scikit-learn is not required for nearest-neighbour spacing.

1.6 Supported Data Formats

Table 1. Supported PointCloudEDA input formats.

Extension	Reader	Expected layout	Notes
.npy	<code>numpy.load</code>	$N \times D, D \geq 3$	Direct NumPy array.
.npz	<code>numpy.load</code>	First array in archive	The first key is used.
.txt	<code>numpy.loadtxt</code>	Whitespace-delimited columns	XYZ followed by optional attributes.
.csv	<code>numpy.loadtxt</code>	Comma-delimited columns	Headerless numeric CSV expected.
.xyz	<code>numpy.loadtxt</code>	Whitespace-delimited XYZ	Additional columns are accepted.
.pts	<code>numpy.loadtxt</code>	Whitespace-delimited points	Additional columns are accepted.
.ply	Internal ASCII parser	ASCII vertex properties	Binary PLY is not handled by the current parser.
.las/.laz	<code>laspy.read</code>	LAS coordinates and attributes	Requires laspy; LAZ may require lazrs.

1.7 Coordinate and Attribute Conventions

PointCloudEDA does not assume a particular coordinate reference system or physical unit. X, Y, and Z are analysed exactly as supplied. Consequently, bounding-box volume and density are only comparable when clouds use compatible units and coordinate conventions.

A fourth column is treated as intensity when present. Colour detection requires at least six columns; the final three columns are interpreted as RGB and normalised to [0,1]. If RGB values exceed 1, they are divided by $\max(255, \text{observed maximum})$.

Note: The current colour detector classifies every cloud with at least six columns as colour-bearing. Ensure the last three columns genuinely represent RGB.

Chapter 2 — Quick Start

2.1 Analyse a Directory

```
from viseda import PointCloudEDA

# Example structure
# pointcloud_dataset/
#   plane/
#     plane.laz
#   vegetation/
#     vegetation_1_3.las

eda = PointCloudEDA(
    verbose=True,
    max_points_per_cloud=200_000,
    compute_neighbors=True,
    compute_geometry=True,
)

eda.load(
    "pointcloud_dataset/",
    label_from_parent=True,
    recursive=True,
)

summary = eda.summary()
print(summary["inventory"])
print(summary["geometry"])
print(summary["quality"])

eda.plot_dataset(
    save_path="outputs_pointcloud/plot_dataset.png"
)
```

2.2 Analyse a Single File

```
from viseda import PointCloudEDA

eda = PointCloudEDA()
eda.load("plane.laz")

rec = eda.get_record(0)
print(rec.n_points)
print(rec.bbox_size)
print(rec.planarity)

eda.plot(
    cloud_index=0,
    save_path="outputs_pointcloud/plot.png",
)
```

2.3 Load NumPy Arrays Directly

```
import numpy as np
from viseda import PointCloudEDA

cloud_a = np.load("bridge.npy")      # shape (N, D)
cloud_b = np.load("road.npy")

eda = PointCloudEDA(
    max_points_per_cloud=100_000,
    compute_neighbors=True,
    compute_geometry=True,
)
eda.load_arrays(
```



```
[cloud_a, cloud_b],
labels=["bridge", "road"],
names=["bridge_001", "road_001"],
)

eda.plot_clouds_grid(n=2, cols=2)
```

2.4 Obtain the Summary

```
stats = eda.summary()

print(stats["inventory"])
print(stats["geometry"])
print(stats["height"])
print(stats["quality"])
print(stats["shape_descriptors"])
print(stats["attributes"])
```

2.5 Generate All Visualisations

```
eda.plot_dataset()                # dataset dashboard
eda.plot(cloud_index=0)           # single-cloud dashboard
eda.plot_clouds_grid(n=12, cols=4) # 3D preview grid
eda.plot_height_distribution()    # overlaid Z distributions
eda.plot_pairwise_cloud_distances() # descriptor-distance heatmap

# Save instead of displaying
eda.plot_dataset(
    save_path="outputs_pointcloud/plot_dataset.png",
    dpi=150,
)
```

2.6 Generate an HTML Report

```
report_path = eda.report(
    "outputs_pointcloud/viseda_pointcloud_report.html"
)
print(report_path)
```

2.7 Command-Line Interface

```
# Full dataset analysis and report
viseda cloud pointcloud_dataset/ --label-from-parent --report outputs_pointcloud/report.html --plot --save-plot
outputs_pointcloud/plot_dataset.png

# Limit clouds and per-cloud points
viseda cloud pointcloud_dataset/ --max-clouds 100 --max-points 200000 --label-from-parent
```

Note: The current CLI exposes `max_clouds` and `max_points_per_cloud`. More specialised constructor options such as `compute_neighbors`, `compute_geometry`, `neighbor_sample_size`, and `duplicate_decimals` are available through the Python API.

2.8 Run the Complete Test Script

```
# Synthetic data only
python testpointcloud.py

# Analyse a directory and save plots
python testpointcloud.py --dir "C:/path/to/pointclouds" --save-plots --quick

# Analyse one file
python testpointcloud.py --file "C:/path/to/cloud.laz" --save-plots
```

Chapter 3 — The PointCloudEDA Class

3.1 Constructor

```
from viseda import PointCloudEDA

eda = PointCloudEDA(
    verbose=True,
    max_clouds=None,
    max_points_per_cloud=200_000,
    sample_seed=0,
    compute_neighbors=True,
    compute_geometry=True,
    neighbor_sample_size=10_000,
    duplicate_decimals=5,
)
```

Table 2. PointCloudEDA constructor parameters.

Parameter	Type/default	Meaning and practical effect
verbose	bool = True	Print loading progress and error messages.
max_clouds	int None = None	Limit the number of clouds analysed.
max_points_per_cloud	int None = 200000	Randomly downsample clouds above this count; None retains all points.
sample_seed	int = 0	Seed used for reproducible downsampling and sampling.
compute_neighbors	bool = True	Compute nearest-neighbour spacing with scikit-learn.
compute_geometry	bool = True	Compute covariance-eigenvalue shape descriptors.
neighbor_sample_size	int = 10000	Maximum points used for nearest-neighbour calculations.
duplicate_decimals	int = 5	XYZ rounding precision used to estimate duplicate points.

3.2 load() — Files and Directories

```
def load( self, source: Union[str, Path, Sequence[Union[str, Path]]], labels: Optional[Dict[str, str]] = None, label_from_parent: bool = False, recursive: bool = True, ) -> "PointCloudEDA":
```

load() resolves a single file, a sequence of files, or a directory. Directory discovery uses the supported extension set and can recurse into subdirectories. If max_clouds is configured, the resolved path list is truncated before analysis.

labels accepts a mapping from path to label. With label_from_parent=True, the immediate parent directory name becomes the label. Each file is read, downsampled if necessary, stored in the internal array cache, and converted into a PointCloudRecord.

3.3 load_arrays() — In-Memory Clouds

```
def load_arrays( self, arrays: Sequence[np.ndarray], labels: Optional[Sequence[str]] = None, names: Optional[Sequence[str]] = None, ) -> "PointCloudEDA":
```

load_arrays() accepts any sequence of $N \times D$ arrays. The optional labels and names sequences are truncated consistently when max_clouds is set. Names default to <array_0>, <array_1>, and so forth. Invalid arrays are retained as corrupt records with their error messages.

3.4 summary() — Nested Dataset Statistics

```
def summary(self) -> Dict[str, Any]:
```

summary() returns inventory, geometry, height, quality, shape_descriptors, and attributes sections. Each scalar distribution uses the same statistics schema: count, mean, standard deviation, minimum, 25th percentile, median, 75th percentile, and maximum.

```
{
  "inventory": {
    "total_clouds", "valid_clouds", "corrupt_clouds",
    "corrupt_paths", "format_distribution",
    "label_distribution", "point_count",
    "dimension_count", "has_color_count",
    "has_intensity_count"
  },
  "geometry": {
    "bbox_volume", "density",
    "span_x", "span_y", "span_z",
    "bbox_size_mean", "xyz_mean_mean", "xyz_std_mean"
  },
  "height": {
    "height_min", "height_max",
    "height_mean", "height_std"
  },
  "quality": {
    "finite_fraction", "duplicate_fraction",
    "outlier_fraction", "nearest_neighbor_mean",
    "nearest_neighbor_median", "nearest_neighbor_std"
  },
  "shape_descriptors": {
    "linearity", "planarity",
    "scattering", "curvature"
  },
  "attributes": {
    "intensity_mean", "intensity_std",
    "rgb_mean_mean", "rgb_std_mean"
  }
}
```

3.5 get_record() and get_cloud()

```
record = eda.get_record(index=0)
cloud = eda.get_cloud(index=0)

print(record.path)
print(record.n_points)
print(cloud.shape)
```

`get_record()` returns the `PointCloudRecord` object. `get_cloud()` returns the loaded and potentially downsampled array associated with the record. For file-backed records, the array is normally served from the internal cache created during loading.

3.6 pairwise_cloud_distances()

```
def pairwise_cloud_distances(self, max_clouds: int = 50) -> Tuple[np.ndarray, List[str]]:
```

This method compares clouds using 15 summary descriptors rather than raw point-to-point Chamfer distance. The feature vector contains point count; X, Y, and Z spans; bounding-box volume; density; height mean and standard deviation; nearest-neighbour mean; duplicate and outlier fractions; and the four PCA shape descriptors.

Every feature is z-score normalised across the selected clouds. Zero standard deviations are replaced by 1, missing or infinite values are converted to 0, and Euclidean distances are calculated in the normalised descriptor space.

$$D_{ij} = \sqrt{\Sigma_k (Z_{ik} - Z_{jk})^2}$$

Z is the z-score-normalised 15-feature cloud descriptor.

Note: The result is not a Chamfer, Hausdorff, Earth Mover's, or ICP alignment distance. It compares global descriptors and is designed for fast dataset-level EDA.

3.7 report()

```
def report(self, output_path: str = "viseda_pointcloud_report.html") -> str:
```

report() writes a self-contained HTML document using light-theme cards and statistic rows. It includes inventory, geometry, height, quality, shape descriptors, and attribute summaries.

Chapter 4 — The PointCloudRecord Object

Every loaded sample produces one PointCloudRecord. The object is intentionally lightweight and uses `__slots__` to define all fields. Fields that cannot be computed because an optional analysis is disabled or a dependency is unavailable are set to `None`.

4.1 Identity and Status Fields

Field	Meaning
path	File path, supplied name, or <array_N> identifier.
label	Class label from the mapping or parent folder.
file_ext	Input extension or 'array'.
file_size_kb	File size in kilobytes for file-backed clouds.
n_points	Number of finite points after downsampling and filtering.
n_dims	Number of columns in the normalised array.
dtype	Internal array dtype.
has_color	Whether the cloud has at least six columns and final RGB-like columns.
has_intensity	True when at least four columns are present.
is_corrupt	Whether loading or analysis failed.
error	Stored exception message for a corrupt record.

4.2 Coordinate and Extent Fields

Field	Shape / type	Meaning
xyz_min	3-vector	Minimum X, Y, Z.
xyz_max	3-vector	Maximum X, Y, Z.
xyz_mean	3-vector	Coordinate mean.
xyz_std	3-vector	Coordinate standard deviation.
centroid	3-vector	Copy of xyz_mean.
bbox_size	3-vector	xyz_max - xyz_min.
span_x/y/z	scalar	Individual bounding-box dimensions.
bbox_volume	scalar	Product of spans with a 1e-12 floor per dimension.
density	scalar	n_points divided by bounding-box volume.

4.3 Height and Radial Fields

Field	Meaning
height_min / max	Minimum and maximum Z.
height_mean / std	Mean and standard deviation of Z.
z_percentiles	Z percentiles at 0, 5, 25, 50, 75, 95, and 100.
radial_distance_mean	Mean Euclidean distance from the centroid.
radial_distance_std	Standard deviation of centroid distances.

4.4 Quality Fields

Field	Meaning
finite_fraction	Fraction of original rows whose XYZ values are all finite.
duplicate_fraction	Fraction lost when rounded XYZ rows are reduced to unique coordinates.
outlier_fraction	Fraction whose centroid distance exceeds mean + 3 standard deviations.
nearest_neighbor_mean	Mean nearest-other-point distance.
nearest_neighbor_median	Median nearest-other-point distance.
nearest_neighbor_std	Variability of nearest-neighbour spacing.

4.5 PCA Shape Descriptors

Field	Formula	Interpretation
linearity	$(\lambda_1 - \lambda_2) / \lambda_1$	High when one principal direction dominates.
planarity	$(\lambda_2 - \lambda_3) / \lambda_1$	High for broad, thin surfaces.
scattering	λ_3 / λ_1	High for volumetric, isotropic point distributions.
curvature	$\lambda_3 / (\lambda_1 + \lambda_2 + \lambda_3)$	Smallest-eigenvalue contribution to total variance.
normal_entropy	$-\sum p_i \log(p_i)$	Entropy of the normalised eigenvalue spectrum.

Note: The implementation computes one global covariance matrix per cloud, not local neighbourhood descriptors at every point. The resulting values summarise overall cloud shape.

4.6 Attribute and Preview Fields

Field	Meaning
intensity_mean / std	Mean and standard deviation of column 3 when present.
rgb_mean / std	Mean and standard deviation of the final three columns after RGB normalisation.
sample_points	Up to 5,000 XYZ points retained for quick plotting.

Chapter 5 — What Is Analysed

5.1 Input Validation and Finite Filtering

The normaliser requires a two-dimensional array with at least three columns. A one-dimensional vector of length at least three is treated as one point. The first quality calculation is `finite_fraction`: the mean of a mask that requires all three XYZ values to be finite.

$$\text{finite_fraction} = N_{\text{finite}} / N_{\text{original}}$$

Rows with invalid XYZ values are removed before all subsequent calculations. Attributes in remaining rows are not required to be finite for the XYZ geometry, although intensity and RGB summaries explicitly filter non-finite attribute values.

5.2 Reproducible Downsampling

When a cloud contains more than `max_points_per_cloud` points, the implementation selects a random subset without replacement using NumPy's `default_rng(sample_seed)`. The chosen indices are sorted before indexing so that the output remains in original row order.

Note: `n_points` refers to the analysed/downsampled cloud, not necessarily the original file point count. Set `max_points_per_cloud=None` to retain all points when memory permits.

5.3 Coordinate Statistics and Centroid

PointCloudEDA calculates minimum, maximum, mean, and standard deviation independently for X, Y, and Z. The centroid equals the arithmetic coordinate mean.

$$\mathbf{c} = (1/N) \sum_i \mathbf{p}_i$$

$\mathbf{p}_i$ is the XYZ coordinate of point i .

5.4 Bounding Box, Volume, and Density

$$\begin{aligned} \text{bbox_size} &= \text{xyz_max} - \text{xyz_min} \\ \text{bbox_volume} &= \prod_a \max(\text{bbox_size}_a, 10^{-12}) \\ \text{density} &= N / \text{bbox_volume} \end{aligned}$$

Density is therefore an axis-aligned bounding-box density, not surface density or neighbourhood density. It is highly sensitive to coordinate units and outliers. A single remote point can enlarge the bounding box and reduce the reported density.

5.5 Height Structure

The Z column is treated as height. PointCloudEDA stores minimum, maximum, mean, standard deviation, and seven percentiles. These summaries are useful for comparing terrain, vegetation, road, object, and building clouds.

Note: If the dataset uses a coordinate system where another axis is vertical, rotate or reorder the coordinates before loading.

5.6 Radial Spread and Outlier Fraction

For each point, PointCloudEDA calculates its Euclidean distance from the centroid. Points whose distance exceeds the mean radial distance plus three radial standard deviations are counted as outliers.

$$\begin{aligned} \text{outlier} &= [r_i > \text{mean}(\mathbf{r}) + 3 \cdot \text{std}(\mathbf{r})] \\ \text{outlier_fraction} &= N_{\text{outlier}} / N \end{aligned}$$

This is a global radial heuristic. It is effective for isolated far-away points but can classify legitimate distant structures as outliers in elongated or multi-component clouds.

5.7 Duplicate Fraction

XYZ coordinates are rounded to `duplicate_decimals` places. The number of unique rounded rows is compared with the analysed point count.

$$\text{duplicate_fraction} = 1 - N_{\text{unique}}(\text{round}(\text{XYZ}, d)) / N$$

The decimal precision determines the spatial tolerance. With `duplicate_decimals=5`, points that differ only beyond five decimal places are treated as duplicates.

5.8 Nearest-Neighbour Spacing

When `compute_neighbors=True`, up to `neighbor_sample_size` points are sampled. scikit-learn `NearestNeighbors` is fitted with `n_neighbors=2`. The first returned neighbour is the point itself; the second distance is the nearest other point.

The mean, median, and standard deviation describe local spacing. Low mean spacing indicates dense sampling. High spacing variability may indicate holes, mixed densities, or multiple spatial components.

5.9 PCA-Based Global Shape

Up to 50,000 sampled XYZ points are centred and used to form a 3×3 covariance matrix. Its eigenvalues are sorted $\lambda_1 \geq \lambda_2 \geq \lambda_3 \geq 0$. The four descriptors capture whether the global point distribution is line-like, plane-like, or volumetric.

$$\begin{aligned}\text{linearity} &= (\lambda_1 - \lambda_2) / \lambda_1 \\ \text{planarity} &= (\lambda_2 - \lambda_3) / \lambda_1 \\ \text{scattering} &= \lambda_3 / \lambda_1 \\ \text{curvature} &= \lambda_3 / (\lambda_1 + \lambda_2 + \lambda_3)\end{aligned}$$

A linear cable, rail, or elongated object tends to have high linearity. A road surface or wall tends to have high planarity. A sphere, foliage cluster, or volumetric object tends to have higher scattering.

5.10 Intensity and RGB Attributes

When $D \geq 4$, column 3 is summarised as intensity. When $D \geq 6$, the final three columns are interpreted as RGB. RGB is normalised to $[0,1]$, and per-channel mean and standard deviation are stored.

5.11 Descriptor-Space Dataset Distance

The pairwise distance matrix compares global cloud summaries rather than raw geometry. This makes it computationally inexpensive and appropriate for broad EDA, but it does not detect rigidly transformed copies or detailed local-shape differences as accurately as point-set distances.

Chapter 6 — Visualisations

6.1 plot_dataset() — Dataset Dashboard

```
def plot_dataset( self, figsize: Tuple[int, int] = (22, 18), save_path: Optional[str] = None, dpi: int = 150, )
-> None:
```

PointCloudEDA — Dataset Analysis

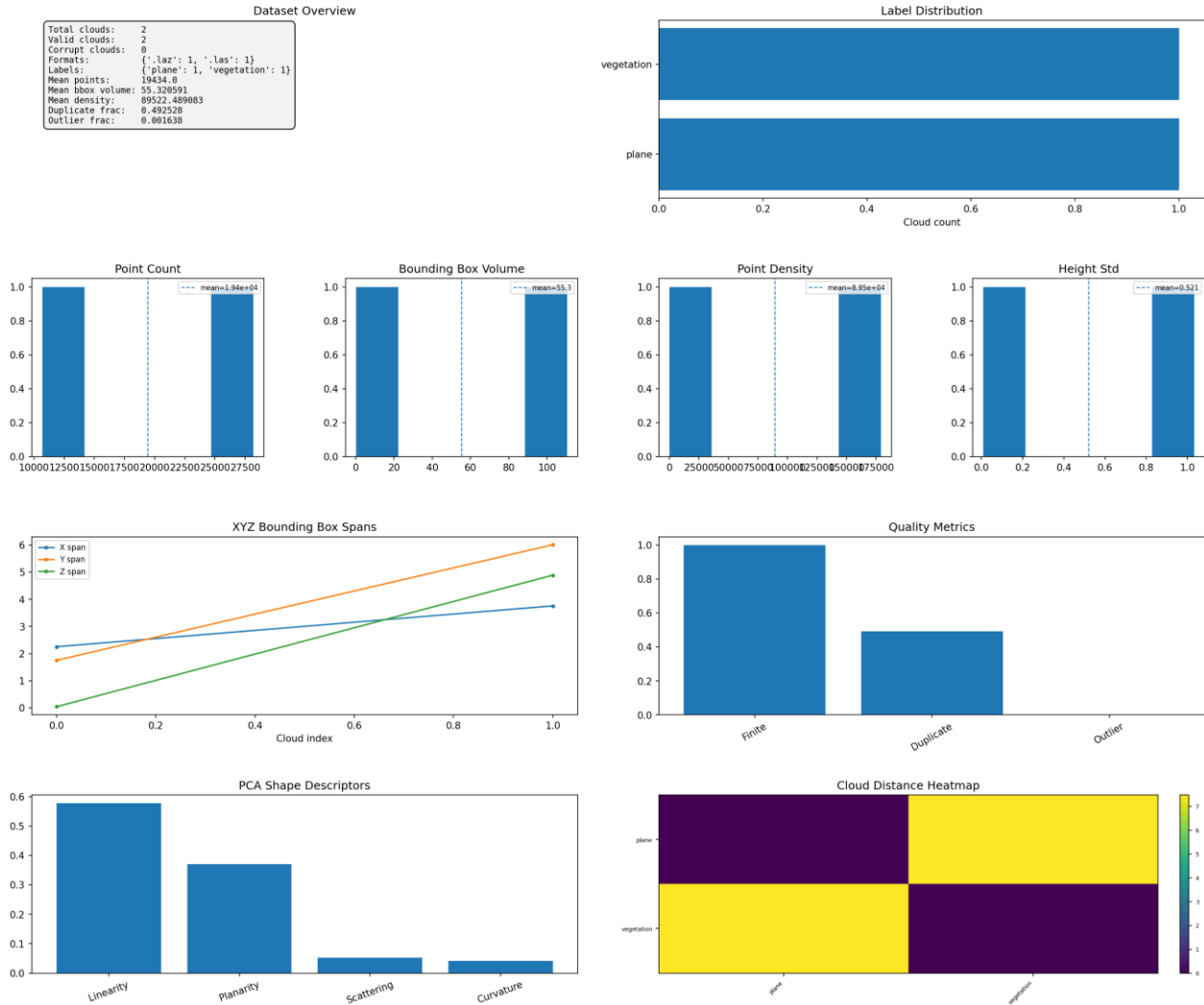


Figure 6-1. PointCloudEDA Dataset Dashboard

The dataset dashboard combines inventory, class balance, point counts, bounding-box volume, density, height variation, XYZ spans, quality indicators, shape descriptors, and pairwise cloud distances.

Row 1: Dataset Overview and Label Distribution

The overview card reports total, valid, and corrupt clouds; format and label counts; mean point count; mean bounding-box volume; mean density; duplicate fraction; and outlier fraction. The label bar chart reveals class balance.

Row 2: Point Count, Volume, Density, and Height Variation

Four histograms show the distributions of analysed point count, axis-aligned bounding-box volume, point density, and Z standard deviation. Dashed vertical lines mark the dataset mean.

Row 3: XYZ Spans and Quality Metrics

The XYZ span plot shows how X, Y, and Z extents change across cloud index. The quality panel reports mean finite fraction, duplicate fraction, outlier fraction, and nearest-neighbour mean when available.

Row 4: PCA Shape Descriptors and Distance Heatmap

The descriptor bars summarise mean linearity, planarity, scattering, and curvature. The heatmap compares up to 30 clouds using the normalised global feature vector.

6.2 plot() — Single-Cloud Dashboard

```
def plot( self, cloud_index: int = 0, max_points: int = 20_000, figsize: Tuple[int, int] = (18, 14), save_path: Optional[str] = None, dpi: int = 150, ) -> None:
```

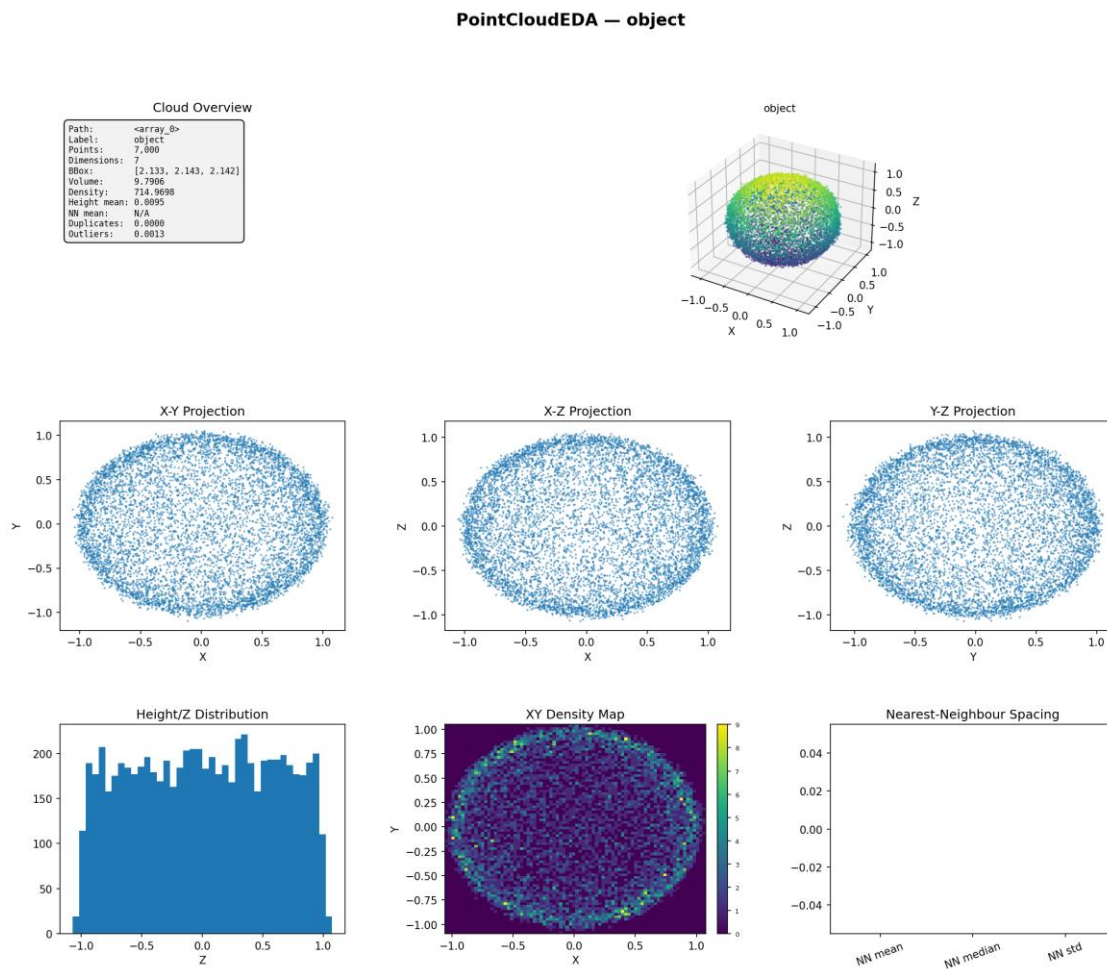


Figure 6-2. Single-Cloud Dashboard

A selected cloud is shown through a metadata card, a 3D scatter plot, XY/XZ/YZ projections, a Z histogram, an XY density map, and nearest-neighbour spacing bars.

Cloud Overview Card

The card displays path, label, point count, dimensionality, bounding-box size, volume, density, mean height, nearest-neighbour mean, duplicate fraction, and outlier fraction.

3D Scatter and Orthographic Projections

The 3D panel colours points by Z. The XY, XZ, and YZ projections reveal planarity, vertical structure, elongation, and hidden overlap that may be difficult to assess in a perspective view.

Height Histogram and XY Density Map

The Z histogram exposes layers and height modes. The XY density map uses a two-dimensional histogram and is especially useful for detecting coverage gaps and uneven sampling.

Nearest-Neighbour Spacing

The final panel compares the mean, median, and standard deviation of nearest-neighbour distances. When neighbour computation is disabled or unavailable, values are displayed as zero in this plotting helper.

6.3 plot_clouds_grid() — Preview Grid

```
def plot_clouds_grid( self, n: int = 12, cols: int = 4, max_points: int = 8_000, figsize: Optional[Tuple[int, int]] = None, save_path: Optional[str] = None, dpi: int = 150, ) -> None:
```

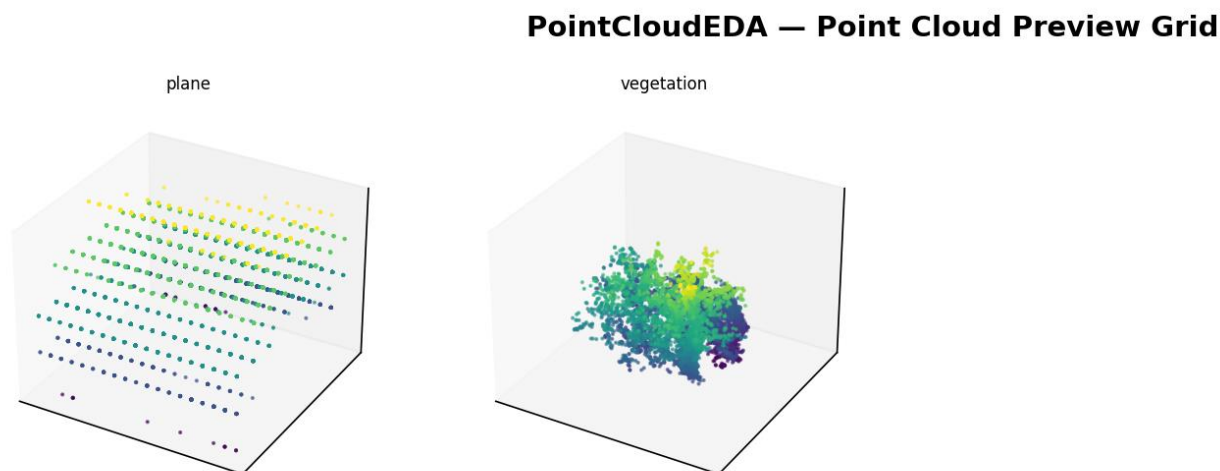


Figure 6-3. Point Cloud Preview Grid

Middle-level visual inspection across several clouds. Each panel uses a sampled 3D scatter coloured by Z and titled with the cloud label or filename.

Use this grid as a sanity check for orientation, scale, content, and label correctness. Large differences in axis scale may make some clouds appear compressed; compare the numeric spans in the dataset dashboard.

6.4 plot_height_distribution() — Z Distribution Overlay

```
def plot_height_distribution( self, figsize: Tuple[int, int] = (12, 6), save_path: Optional[str] = None, dpi: int = 150, ) -> None:
```

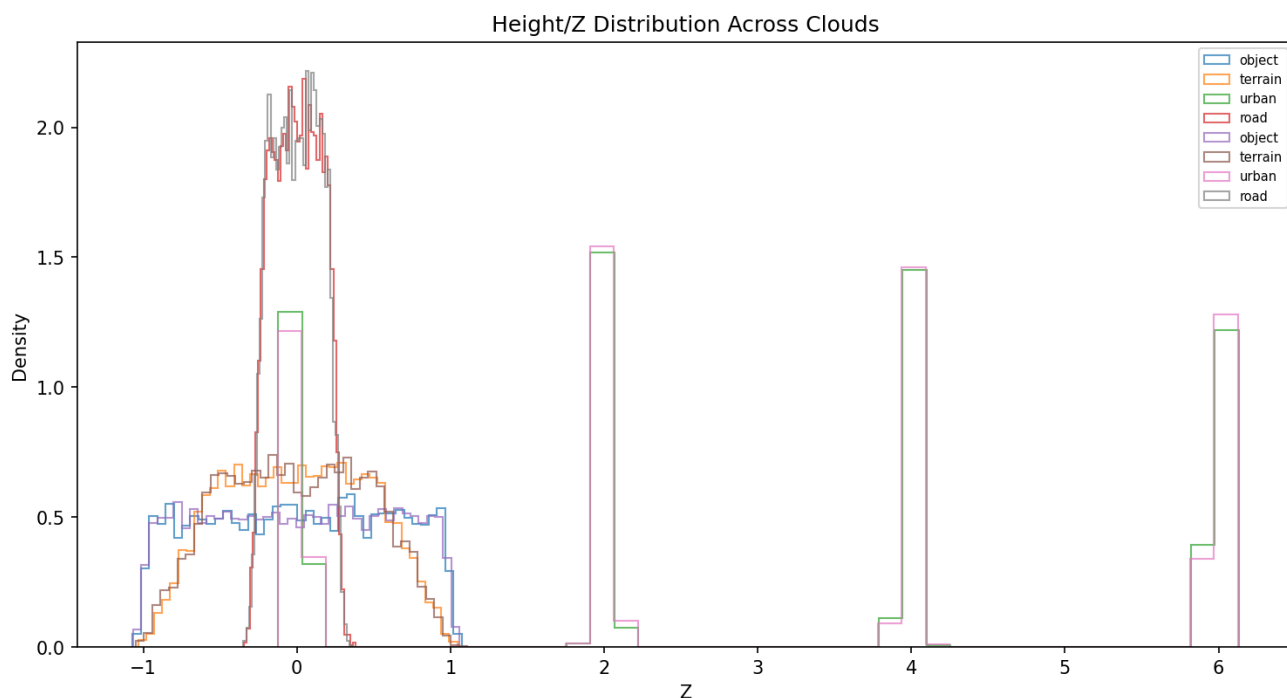


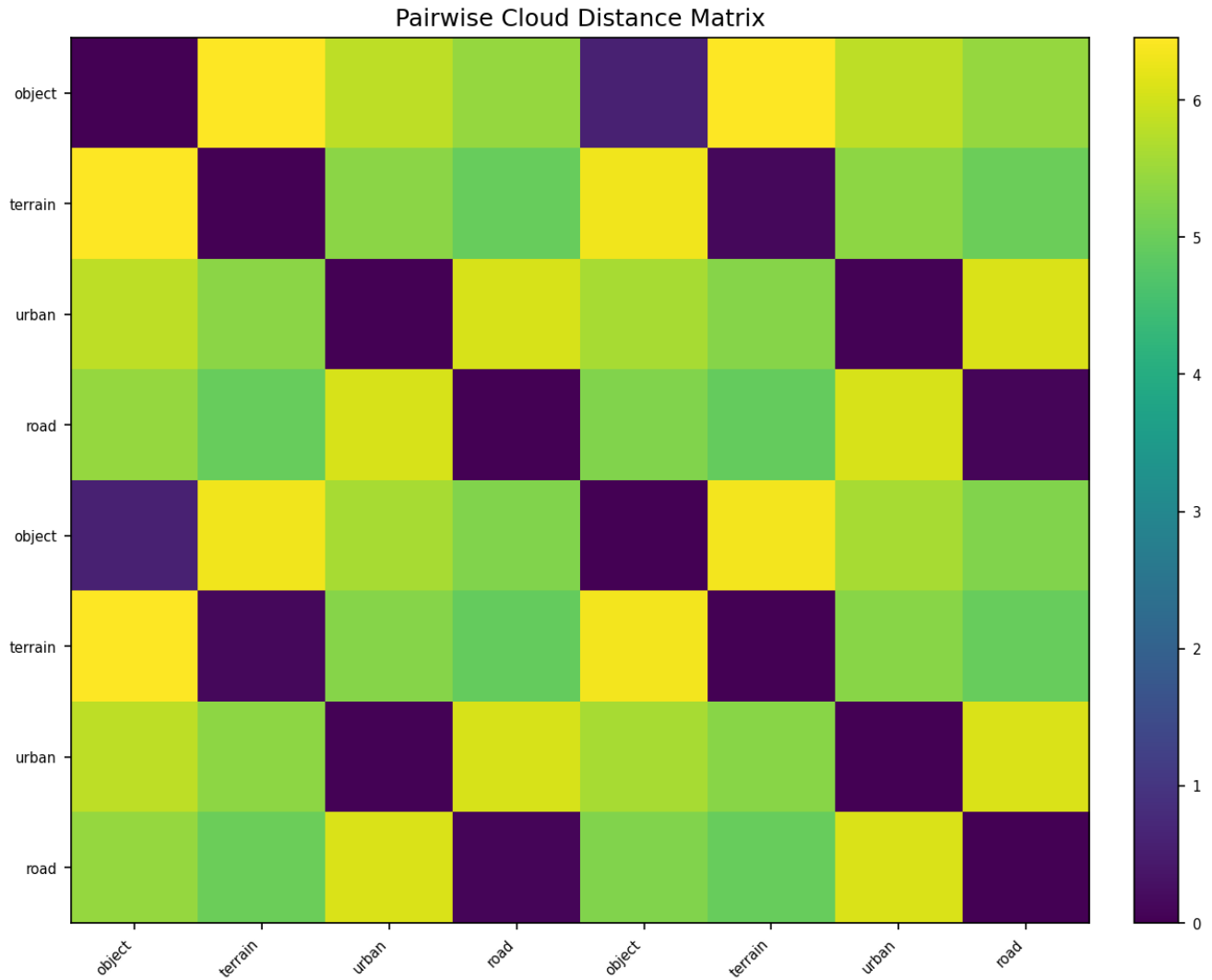
Figure 6-4. Height/Z Distributions Across Clouds

Overlaid density-normalised Z histograms for up to 30 valid clouds. The plot reveals layers, terrain profiles, object heights, and between-class vertical differences.

Legends are displayed when there are at most ten valid clouds. Repeated labels in a synthetic test reflect multiple clouds per class; in large datasets the overlay is intended for pattern recognition rather than individual curve identification.

6.5 `plot_pairwise_cloud_distances()` — Distance Heatmap

```
def plot_pairwise_cloud_distances( self, max_clouds: int = 50, figsize: Tuple[int, int] = (10, 8), save_path:
Optional[str] = None, dpi: int = 150, ) -> None:
```

Figure 5. Pairwise Cloud Distance Matrix

A symmetric matrix of Euclidean distances in the normalised 15-feature descriptor space. The diagonal is zero; darker off-diagonal cells indicate globally similar clouds.

Repeated block or checkerboard patterns often indicate that multiple clouds share the same synthetic scene type or class geometry. Use the matrix to identify redundancy before creating train/validation/test splits.

Chapter 7 — Interpreting Results and Common Findings

7.1 Red Flags to Review

Finding	Why it matters	Recommended investigation
finite_fraction < 1	Rows contained NaN or infinite XYZ coordinates.	Identify the source file and quantify removed rows.
High duplicate_fraction	Point count is inflated without adding spatial information.	Inspect acquisition/export process and rounding precision.
High outlier_fraction	Isolated points may distort scale and density.	Visualise projections and consider robust filtering.
Very low density	Sparse sampling or an expanded bounding box.	Check coordinate units and remote outliers.
Very high density	Possible duplicate points or very small coordinate extent.	Compare duplicate fraction and units.
Large NN std	Strongly uneven local sampling.	Inspect density maps and acquisition overlap.
Mixed point counts	Batching and model input sizes may be inconsistent.	Downsample, voxelise, or use dynamic batching.
Mixed attribute counts	Some clouds include intensity/RGB while others do not.	Standardise the feature schema before training.
Distance heatmap dark clusters	Near-redundant clouds may cause leakage.	Group similar samples before splitting.
Unexpected shape descriptors	Labels may not match geometry.	Review cloud orientation and class assignment.

7.2 Reading Shape Descriptors Together

Typical global structure	Expected descriptor pattern
Line or elongated cable/rail	High linearity; low planarity and scattering.
Plane, wall, road, roof	High planarity; low scattering and curvature.
Sphere, foliage, volumetric object	Higher scattering; lower linearity and planarity.
Layered building cloud	Descriptor values depend on total extent; height histogram often shows multiple modes.
Noisy isotropic cluster	Moderate/high scattering and entropy; potentially high outlier fraction.

7.3 Interpreting Density Carefully

Because density uses the axis-aligned bounding-box volume, comparisons are meaningful only when coordinates use compatible units, and clouds cover comparable regions. A road corridor and a compact object can have very different density values even if their sensors have the same physical sampling resolution.

7.4 Example Findings from the Attached Outputs

The attached single-cloud object dashboard shows a roughly spherical structure: XY, XZ, and YZ projections are similarly rounded, and the 3D scatter has comparable extent on all axes. This visual pattern is consistent with a volumetric object rather than a plane or line.

The two-cloud dataset dashboard contrasts a plane and vegetation cloud. The span curves and point-density panels show that their geometric scale and sampling differ substantially. The pairwise heatmap has a large off-diagonal distance, confirming that their global descriptors are distinct.

The eight-cloud distance matrix exhibits repeated dark cells for same-type synthetic clouds and bright cells between structurally different types. This demonstrates the intended use of descriptor-space distance for rapid dataset similarity screening.

7.5 Thresholds Are Dataset-Dependent

Note: PointCloudEDA does not enforce universal quality thresholds. A nearest-neighbour distance of 0.05 may be dense in metres and sparse in millimetres. Interpret distributions relative to the dataset, acquisition system, coordinate units, and downstream task.

Chapter 8 — Advanced Usage

8.1 Working with Large Datasets

```
eda = PointCloudEDA(
    max_clouds=500,
    max_points_per_cloud=100_000,
    compute_neighbors=False,    # fastest quick overview
    compute_geometry=True,
)
eda.load("large_dataset/", label_from_parent=True)
```

For a first audit, limit clouds and disable neighbour calculations. Geometry descriptors use at most 50,000 points internally. After identifying suspicious classes or files, rerun those subsets with `compute_neighbors=True` and a larger point cap.

8.2 Access and Filter Records

```
valid = [r for r in eda._records if not r.is_corrupt]

# Highest duplicate fractions
duplicates = sorted(
    valid,
    key=lambda r: r.duplicate_fraction or 0.0,
    reverse=True,
)[:10]

# Most planar clouds
planar = sorted(
    valid,
    key=lambda r: r.planarity or 0.0,
    reverse=True,
)[:10]

for rec in planar:
    print(rec.path, rec.planarity, rec.n_points)
```

8.3 Export Records to pandas

```
import pandas as pd

rows = []
for r in eda._records:
    if r.is_corrupt:
        continue
    rows.append({
        "path": r.path,
        "label": r.label,
        "n_points": r.n_points,
        "bbox_volume": r.bbox_volume,
        "density": r.density,
        "height_std": r.height_std,
        "duplicate_fraction": r.duplicate_fraction,
        "outlier_fraction": r.outlier_fraction,
        "nn_mean": r.nearest_neighbor_mean,
        "linearity": r.linearity,
        "planarity": r.planarity,
        "scattering": r.scattering,
        "curvature": r.curvature,
    })

df = pd.DataFrame(rows)
df.to_csv("pointcloud_eda_records.csv", index=False)
```


8.4 Use PointCloudEDA Before Model Training

```
# Example preprocessing decisions driven by EDA
stats = eda.summary()

# Choose a common point budget from the median analysed count
median_points = int(
    stats["inventory"]["point_count"]["median"]
)

# Flag clouds for review
review = [
    r.path for r in eda._records
    if not r.is_corrupt and (
        (r.finite_fraction or 0) < 1.0
        or (r.duplicate_fraction or 0) > 0.20
        or (r.outlier_fraction or 0) > 0.05
    )
]
```

Note: The numerical cut-offs in this example are illustrative workflow choices, not thresholds implemented or recommended universally by PointCloudEDA.

8.5 Prevent Similarity Leakage

Use `pairwise_cloud_distances()` before assigning random splits. Build groups of clouds with very small descriptor distances, then ensure every group is assigned wholly to one split. For high-stakes duplicate detection, supplement the descriptor matrix with registration-aware metrics such as Chamfer distance after alignment.

8.6 Compare Two Datasets

```
train = PointCloudEDA(compute_neighbors=False)
train.load("train/", label_from_parent=True)

test = PointCloudEDA(compute_neighbors=False)
test.load("test/", label_from_parent=True)

train_stats = train.summary()
test_stats = test.summary()

print("Train density:",
      train_stats["geometry"]["density"]["mean"])
print("Test density:",
      test_stats["geometry"]["density"]["mean"])
print("Train planarity:",
      train_stats["shape_descriptors"]["planarity"]["mean"])
print("Test planarity:",
      test_stats["shape_descriptors"]["planarity"]["mean"])
```

8.7 Reproducibility Considerations

Keep `sample_seed` constant when comparing runs. Record `max_points_per_cloud`, `neighbor_sample_size`, `compute_neighbors`, `compute_geometry`, and `duplicate_decimals` with experimental metadata. Changing these values can alter record statistics and pairwise distances.

Chapter 9 — HTML Report, CLI, and Testing

9.1 HTML Report Structure

The self-contained HTML report uses responsive cards, badges, bars, and statistic rows. It contains inventory counts; label and format distributions; point and dimension counts; geometry; height; quality; shape descriptors; and intensity/RGB attribute summaries. No external JavaScript or CSS files are required.

9.2 Command-Line Options

Option	Meaning
path	Point-cloud file or dataset directory.
--max-clouds	Maximum number of clouds to analyse.
--max-points	Maximum analysed points per cloud; CLI default is 1,000,000.
--label-from-parent	Use parent directory as class label.
--report	Write the HTML report.
--plot	Generate the dataset dashboard.
--save-plot	Save the dashboard to a specified path.

9.3 Test Coverage

The supplied testpointcloud.py validates array loading, directory loading, NPY/NPZ/TXT/XYZ/PLY formats, summary fields, PointCloudRecord completeness, get_record(), get_cloud(), pairwise distances, all five visualisations, HTML reporting, max_clouds, all-nonfinite inputs, and single-point clouds.

```
python testpointcloud.py --save-plots

# Quick mode skips slower nearest-neighbour calculations
python testpointcloud.py --save-plots --quick

# Real LAS/LAZ directory
python testpointcloud.py --dir "C:/Users/Desktop/testptcloud" --save-plots --quick
```

9.4 Common Errors

Error / symptom	Cause	Resolution
No valid point clouds loaded	Unsupported files, corrupt arrays, or missing LAS reader.	Check extensions and install laspy/lazrs.
Point cloud array must have shape (N,D)	Input is not two-dimensional or has fewer than three columns.	Reshape and ensure XYZ are first.
Nearest-neighbour panel empty	compute_neighbors=False, one point, or scikit-learn unavailable.	Enable neighbours and install scikit-learn.
Binary PLY fails	Current internal PLY reader expects ASCII.	Convert to ASCII PLY, NPY, LAS, or extend the reader.
Density values appear extreme	Units differ, or bounding box is affected by outliers.	Standardise coordinates and inspect projections.
Output DOCX/PDF permission denied	The output file is open in Word or a PDF viewer.	Close it or allow the generator to use its timestamped fallback.

Chapter 10 — API Reference

Method	Current signature	Purpose
<code>__init__</code>	<code>PointCloudEDA(verbose=True, max_clouds=None, max_points_per_cloud=200000, sample_seed=0, compute_neighbors=True, compute_geometry=True, neighbor_sample_size=10000, duplicate_decimals=5)</code>	Create an analysis session.
<code>load</code>	<code>def load(self, source: Union[str, Path, Sequence[Union[str, Path]]], labels: Optional[Dict[str, str]] = None, label_from_parent: bool = False, recursive: bool = True,) -> "PointCloudEDA":</code>	Load one file, a list, or a directory.
<code>load_arrays</code>	<code>def load_arrays(self, arrays: Sequence[np.ndarray], labels: Optional[Sequence[str]] = None, names: Optional[Sequence[str]] = None,) -> "PointCloudEDA":</code>	Load $N \times D$ arrays.
<code>summary</code>	<code>def summary(self) -> Dict[str, Any]:</code>	Return nested aggregate statistics.
<code>get_record</code>	<code>def get_record(self, index: int = 0) -> PointCloudRecord:</code>	Return one PointCloudRecord.
<code>get_cloud</code>	<code>def get_cloud(self, index: int = 0) -> np.ndarray:</code>	Return one analysed/downsampled array.
<code>pairwise_cloud_distances</code>	<code>def pairwise_cloud_distances(self, max_clouds: int = 50) -> Tuple[np.ndarray, List[str]]:</code>	Return descriptor distance matrix and names.
<code>plot_dataset</code>	<code>def plot_dataset(self, figsize: Tuple[int, int] = (22, 18), save_path: Optional[str] = None, dpi: int = 150,) -> None:</code>	Generate dataset dashboard.
<code>plot</code>	<code>def plot(self, cloud_index: int = 0, max_points: int = 20_000, figsize: Tuple[int, int] = (18, 14), save_path: Optional[str] = None, dpi: int = 150,) -> None:</code>	Generate single-cloud dashboard.
<code>plot_clouds_grid</code>	<code>def plot_clouds_grid(self, n: int = 12, cols: int = 4, max_points: int = 8_000, figsize: Optional[Tuple[int, int]] = None, save_path: Optional[str] = None, dpi: int = 150,) -> None:</code>	Generate multi-cloud preview grid.
<code>plot_height_distribution</code>	<code>def plot_height_distribution(self, figsize: Tuple[int, int] = (12, 6), save_path: Optional[str] = None, dpi: int = 150,) -> None:</code>	Overlay Z distributions.
<code>plot_pairwise_cloud_distances</code>	<code>def plot_pairwise_cloud_distances(self, max_clouds: int = 50, figsize: Tuple[int, int] = (10, 8), save_path: Optional[str] = None, dpi: int = 150,) -> None:</code>	Plot descriptor-distance matrix.
<code>report</code>	<code>def report(self, output_path: str = "visedata_pointcloud_report.html") -> str:</code>	Write HTML report and return its path.

10.1 Return and Failure Behaviour

`load()` and `load_arrays()` return self, allowing method chaining. `summary()` returns a dictionary. `get_record()` returns a `PointCloudRecord`. `get_cloud()` returns a NumPy array. `pairwise_cloud_distances()` returns (distance_matrix, names). Plotting methods return None. `report()` returns the output path string.

`summary()`, plotting methods, record access, and `report()` require a loaded object and raise `RuntimeError` before `load()`. Pairwise distance analysis requires at least two valid clouds and raises `ValueError` otherwise.

Chapter 11 —Roadmap

Version 1.0.0

- Unified PointCloudEDA class for file and array inputs.
- Dataset summary with inventory, geometry, height, quality, shape, and attribute sections.
- Support for NPY, NPZ, text, XYZ, PTS, ASCII PLY, LAS, and LAZ.
- Reproducible per-cloud downsampling.
- Nearest-neighbour spacing and PCA shape descriptors.
- Dataset dashboard, single-cloud dashboard, preview grid, height overlay, and distance heatmap.
- Styled self-contained HTML report.
- Complete synthetic and file-format test suite.

Roadmap

- Binary PLY and richer PLY property support.
- Coordinate reference system and unit metadata.
- Voxel-density and local neighbourhood density analysis.
- Local surface normals and per-point curvature distributions.
- Ground/non-ground separation and canopy metrics for LiDAR.
- Chamfer, Hausdorff, and Earth Mover's distance options.
- Rigid registration and duplicate detection invariant to translation and rotation.
- Class-conditional dashboards and per-class descriptor comparison.
- Interactive 3D HTML previews.
- Octree, voxel, and occupancy-grid summaries.
- Additional LAS dimensions and waveform metadata.
- Streaming analysis for extremely large LAS/LAZ files.

Appendix A — PointCloudRecord Fields

Field	Group	Meaning
path	Identity	Path/name identifier.
label	Identity	Class label.
file_ext	Identity	Input extension or array.
file_size_kb	Identity	File size in KB.
n_points	Inventory	Analysed finite point count.
n_dims	Inventory	Array column count.
dtype	Inventory	Internal dtype.
has_color	Attributes	RGB-like final columns detected.
has_intensity	Attributes	At least four columns.
is_corrupt	Status	Whether analysis failed.
error	Status	Failure message.
xyz_min	Geometry	Minimum XYZ.
xyz_max	Geometry	Maximum XYZ.
xyz_mean	Geometry	Mean XYZ.
xyz_std	Geometry	XYZ standard deviations.
bbox_size	Geometry	Axis-aligned spans.
bbox_volume	Geometry	Product of spans.
centroid	Geometry	Mean XYZ.
span_x/y/z	Geometry	Individual extents.
density	Geometry	Points per bounding-box volume.
height_min/max	Height	Z extrema.
height_mean/std	Height	Z mean and spread.
z_percentiles	Height	0/5/25/50/75/95/100 percentiles.
radial_distance_mean/std	Quality	Centroid-distance statistics.
nearest_neighbor_mean	Quality	Mean nearest-other-point spacing.
nearest_neighbor_median	Quality	Median nearest spacing.
nearest_neighbor_std	Quality	Spacing variability.
outlier_fraction	Quality	Radial 3-sigma fraction.
duplicate_fraction	Quality	Rounded duplicate estimate.
finite_fraction	Quality	Fraction with finite XYZ.
planarity	Shape	Global PCA planarity.
linearity	Shape	Global PCA linearity.
scattering	Shape	Global PCA scattering.
curvature	Shape	Global PCA curvature.
normal_entropy	Shape	Eigenvalue entropy.
intensity_mean/std	Attributes	Column-3 statistics.
rgb_mean/std	Attributes	Normalised RGB statistics.
sample_points	Preview	Up to 5,000 sampled XYZ points.

Appendix B — summary() Schema

```
{
  "inventory": {
    "total_clouds", "valid_clouds", "corrupt_clouds",
    "corrupt_paths", "format_distribution",
    "label_distribution", "point_count",
    "dimension_count", "has_color_count",
    "has_intensity_count"
  },
  "geometry": {
    "bbox_volume", "density",
    "span_x", "span_y", "span_z",
    "bbox_size_mean", "xyz_mean_mean", "xyz_std_mean"
  },
  "height": {
    "height_min", "height_max",
    "height_mean", "height_std"
  },
  "quality": {
    "finite_fraction", "duplicate_fraction",
    "outlier_fraction",
    "nearest_neighbor_mean",
    "nearest_neighbor_median",
    "nearest_neighbor_std"
  },
  "shape_descriptors": {
    "linearity", "planarity",
    "scattering", "curvature"
  },
  "attributes": {
    "intensity_mean", "intensity_std",
    "rgb_mean_mean", "rgb_std_mean"
  }
}
```

Appendix C — Mathematical Reference

Metric	Definition used by PointCloudEDA
Centroid	$c = (1/N) \sum p_i$
Bounding-box size	$\max(XYZ) - \min(XYZ)$
Bounding-box volume	$\prod \max(\text{span}, 10^{-12})$
Density	$N / \text{bounding-box volume}$
Radial outlier rule	$r_i > \text{mean}(r) + 3 \cdot \text{std}(r)$
Duplicate fraction	$1 - N_{\text{unique}}(\text{round}(XYZ, d)) / N$
Linearity	$(\lambda_1 - \lambda_2) / \lambda_1$
Planarity	$(\lambda_2 - \lambda_3) / \lambda_1$
Scattering	λ_3 / λ_1
Curvature	$\lambda_3 / (\lambda_1 + \lambda_2 + \lambda_3)$
Eigenvalue entropy	$-\sum p_i \log(p_i)$
Descriptor distance	Euclidean distance after feature z-score normalisation

Appendix D — Glossary

Term	Definition
Axis-aligned bounding box	Smallest XYZ-aligned box containing the cloud.
Centroid	Arithmetic mean of all XYZ coordinates.
Chamfer distance	Point-set distance not used by the current pairwise method.
Density	Number of points divided by bounding-box volume in this implementation.
Duplicate point	A rounded XYZ coordinate that appears more than once.
Finite point	A point whose X, Y, and Z are not NaN or infinite.
Intensity	Scalar return-strength or sensor attribute, treated as column 3.
LAS/LAZ	Common LiDAR formats; LAZ is compressed LAS.
Nearest neighbour	Closest other sampled point in Euclidean XYZ space.
Outlier	Point beyond the global radial mean plus three standard deviations.
Planarity	PCA eigenvalue descriptor for plane-like structure.
Point cloud	Unordered set of spatial points with optional attributes.
Scattering	PCA descriptor indicating volumetric spread.
Voxelisation	Discretising 3D space into regular cells.

Appendix E — Acknowledgements

PointCloudEDA builds on NumPy, Matplotlib, scikit-learn, laspy, and python-docx. The module is intended for researchers and practitioners working with LiDAR, photogrammetry, robotics, autonomous systems, structural inspection, terrain modelling, urban mapping, forestry, and 3D object recognition.